LAB 5 · LECTURE 5

Lecture 5 Lab: Same Data, Different Systems

Vector Databases Course | Estimated time: 75 minutes (plus 20 optional) | Platform: Google Colab (free)

Goal

You will load the same 30 documents into a library (FAISS) and a database (Qdrant), and compare what each gives you: metadata, filtering, updates and persistence. You will also fuse two rankings by hand, and optionally repeat the exercise inside PostgreSQL with pgvector.

- Use FAISS for exact search, and notice what it does not store.
- Create a Qdrant collection, add payloads, filter, update and delete.
- Show that post-filtering can return fewer than k results, and filter-aware search does not.
- Persist and reload both systems.
- Implement reciprocal rank fusion for hybrid search.
- Fill in a scorecard and justify a choice for three scenarios.

Setup

Create a new Colab notebook and run each block in its own cell, in order.

Cell 1: install and import

```
!pip install -q qdrant-client faiss-cpu sentence-transformers pandas numpy

import re, os, shutil
import numpy as np, pandas as pd
import faiss
from qdrant_client import QdrantClient, models
from sentence_transformers import SentenceTransformer
```

Part 1: Data and embeddings

The same thirty help-center titles as Lecture 3, now with extra fields (topic, year) that a real system would store as metadata.

Cell 2: corpus and embeddings

```
data = [
 ("account", "How to reset your account password"),
 ("account", "Change the email address on my profile"),
 ("account", "Delete my account permanently"),
 ("account", "Enable two-factor authentication"),
 ("account", "I forgot my login credentials"),
 ("account", "Update my display name"),
 ("billing", "Why was I charged twice this month"),
 ("billing", "Download my invoice as a PDF"),
 ("billing", "Update my credit card details"),
 ("billing", "How to cancel my subscription"),
 ("billing", "Refund policy for cancelled orders"),
 ("billing", "Apply a discount code at checkout"),
 ("shipping", "Where is my package right now"),
 ("shipping", "How long does international delivery take"),
 ("shipping", "Change the delivery address after ordering"),
 ("shipping", "Shipping costs for large items"),
 ("shipping", "My parcel arrived damaged"),
 ("shipping", "Track an order without an account"),
 ("technical", "The app crashes when I open settings"),
 ("technical", "Wi-Fi keeps disconnecting every few minutes"),
 ("technical", "Laptop battery drains too quickly"),
 ("technical", "My screen flickers after the update"),
 ("technical", "How to back up my phone"),
 ("technical", "Bluetooth headphones will not pair"),
 ("returns", "Start a return for a defective item"),
 ("returns", "How to exchange a product for a different size"),
 ("returns", "Is a return label free"),
 ("returns", "Return window for electronics"),
 ("returns", "Where to drop off my return"),
 ("returns", "Refund timeline after a return"),
]
topics = [t for t, _ in data]
texts  = [s for _, s in data]
years  = [2022 + i % 4 for i in range(len(texts))]

model = SentenceTransformer("all-MiniLM-L6-v2")
D = model.encode(texts, normalize_embeddings=True)
print(D.shape)
```

Part 2: FAISS, the library

Cell 3: index, search, save and reload

```
index = faiss.IndexFlatIP(384)      # inner product = cosine for unit vectors
index.add(D)

def faiss_search(query, k=5):
    qv = model.encode(query, normalize_embeddings=True)
    scores, ids = index.search(qv[None, :], k)
    return pd.DataFrame({"id": ids[0], "score": scores[0].round(3),
                         "text": [texts[i] for i in ids[0]]})

print(faiss_search("I can't sign in to my profile"))

faiss.write_index(index, "help.faiss")
print("reloaded vectors:", faiss.read_index("help.faiss").ntotal)
```

YOUR TURN Look at the result: which columns does FAISS give you, and which did you have to add yourself (text, topic, year)? Where would they live in a real application?

Part 3: Qdrant, the database

Qdrant's local mode runs in memory with no server, which is perfect for learning the API. It is meant for development: use Docker or Qdrant Cloud for performance tests.

Cell 4: create a collection and upload points

```
client = QdrantClient(":memory:")
client.create_collection(
    "help",
    vectors_config=models.VectorParams(size=384, distance=models.Distance.COSINE),
)
points = [
    models.PointStruct(id=i, vector=D[i].tolist(),
                       payload={"text": texts[i], "topic": topics[i], "year": years[i]})
    for i in range(len(texts))
]
client.upsert("help", points=points)
print("points stored:", client.count("help").count)
```

Cell 5: query, filter, read, update, delete

```
qv = model.encode("I can't sign in to my profile", normalize_embeddings=True).tolist()

res = client.query_points("help", query=qv, limit=3, with_payload=True)
for p in res.points:
    print(p.id, round(p.score, 3), p.payload["topic"], "|", p.payload["text"])

flt = models.Filter(must=[
    models.FieldCondition(key="topic", match=models.MatchValue(value="billing")),
    models.FieldCondition(key="year", range=models.Range(gte=2024)),
])
res = client.query_points("help", query=qv, query_filter=flt,
                          limit=5, with_payload=True)
print("filtered:", [(p.id, p.payload["year"]) for p in res.points])

print(client.retrieve("help", ids=[0])[0].payload)
client.set_payload("help", payload={"reviewed": True}, points=[0])
page, next_offset = client.scroll("help", limit=3, with_payload=True)
print([p.payload.get("reviewed") for p in page])

client.upsert("help", points=[models.PointStruct(id=1000, vector=D[0].tolist(),
                                                 payload={"text": "temporary"})])
client.delete("help", points_selector=models.PointIdsList(points=[1000]))
print("after delete:", client.count("help").count)
```

YOUR TURN Which of these operations (filter by topic and year, update a field, delete one item) had no counterpart in the FAISS code? How would you do each one with FAISS alone?

Part 4: Post-filter versus filter-aware search

Cell 6: how many results survive?

```
def faiss_post_filter(query, topic, k=10):
    qv = model.encode(query, normalize_embeddings=True)
    _, ids = index.search(qv[None, :], k)
    return [int(i) for i in ids[0] if topics[i] == topic]

def qdrant_filtered(query, topic, k=10):
    qv = model.encode(query, normalize_embeddings=True).tolist()
    flt = models.Filter(must=[models.FieldCondition(
        key="topic", match=models.MatchValue(value=topic))])
    res = client.query_points("help", query=qv, query_filter=flt, limit=k)
    return [p.id for p in res.points]

tests = [("where is my order", "billing"), ("reset my password", "returns"),
         ("battery problems", "billing"), ("send back an item", "technical")]
for q, t in tests:
    print(f"{q!r} in {t!r}: post-filter={len(faiss_post_filter(q, t))}"
          f"  filter-aware={len(qdrant_filtered(q, t))}")
```

YOUR TURN Explain the numbers. With k = 10, how many results does each approach return, and why? What would you do in FAISS to get 10 matching results?

Part 5: Persistence

Cell 7: store to disk and reopen

```
if os.path.exists("qdrant_local"):
    shutil.rmtree("qdrant_local")

disk = QdrantClient(path="qdrant_local")
disk.create_collection("help", vectors_config=models.VectorParams(
    size=384, distance=models.Distance.COSINE))
disk.upsert("help", points=points)
disk.close()

again = QdrantClient(path="qdrant_local")
print("reopened:", again.count("help").count, "points,",
      again.retrieve("help", ids=[5])[0].payload["text"])
```

YOUR TURN Compare with Cell 3: what does Qdrant's reopened collection contain that the reloaded FAISS index does not?

Part 6: Hybrid search by hand

Many systems fuse a keyword ranking and a vector ranking. Reciprocal rank fusion (RRF) needs only ranks, so the two score scales never have to match.

Cell 8: keyword rank, vector rank, fusion

```
def tokens(t):
    return set(re.findall(r"[a-z0-9']+", t.lower()))

def keyword_rank(query, k=10):
    q = tokens(query)
    scores = np.array([len(q & tokens(t)) for t in texts])
    order = np.argsort(-scores, kind="stable")[:k]
    return [int(i) for i in order if scores[i] > 0]

def vector_rank(query, k=10):
    qv = model.encode(query, normalize_embeddings=True)
    return [int(i) for i in np.argsort(-(D @ qv))[:k]]

def rrf(rank_lists, k=60):
    scores = {}
    for ranks in rank_lists:
        for pos, doc in enumerate(ranks, start=1):
            scores[doc] = scores.get(doc, 0) + 1 / (k + pos)
    return sorted(scores, key=scores.get, reverse=True)

query = "refund for a cancelled order"
lists = {"keyword": keyword_rank(query), "vector": vector_rank(query)}
lists["fused"] = rrf([lists["keyword"], lists["vector"]])
for name, ids in lists.items():
    print(name, [texts[i] for i in ids[:3]])
```

YOUR TURN Try three queries: one with exact wording ('Return window for electronics'), one paraphrased, and one containing a code like 'A-4471'. Which ranking wins in each case, and does fusion help?

Part 7 (optional): pgvector in PostgreSQL

This part installs PostgreSQL inside Colab and builds the pgvector extension from source. It takes about 5 minutes. The version tag may need updating: check the pgvector releases page.

Cell 9: install PostgreSQL and pgvector

```
!apt-get -qq update
!apt-get -qq install -y postgresql postgresql-server-dev-all build-essential git
!git clone -q --branch v0.8.1 https://github.com/pgvector/pgvector.git
!cd pgvector && make -s && make -s install
!service postgresql start
!sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'postgres';"
!pip install -q psycopg2-binary
```

Cell 10: table, index, filtered search

```
import psycopg2
conn = psycopg2.connect(host="localhost", dbname="postgres",
                        user="postgres", password="postgres")
conn.autocommit = True
cur = conn.cursor()
cur.execute("CREATE EXTENSION IF NOT EXISTS vector")
cur.execute("DROP TABLE IF EXISTS help")
cur.execute("CREATE TABLE help (id int PRIMARY KEY, text text, topic text, "
            "year int, embedding vector(384))")

def vec(v):
    return "[" + ",".join(str(float(x)) for x in v) + "]"

for i in range(len(texts)):
    cur.execute("INSERT INTO help VALUES (%s, %s, %s, %s, %s::vector)",
                (i, texts[i], topics[i], years[i], vec(D[i])))
cur.execute("CREATE INDEX ON help USING hnsw (embedding vector_cosine_ops) "
            "WITH (m = 16, ef_construction = 64)")

q = vec(model.encode("I can't sign in to my profile", normalize_embeddings=True))
cur.execute("SELECT text, 1 - (embedding <=> %s::vector) AS score FROM help "
            "WHERE topic = %s ORDER BY embedding <=> %s::vector LIMIT 3",
            (q, "account", q))
for row in cur.fetchall():
    print(row)
```

YOUR TURN In pgvector the filter is an ordinary SQL WHERE clause. Add AND year >= 2024, and run EXPLAIN on the query. With only 30 rows the planner will probably scan the table: why? What would change with a million rows?

Part 8: Scorecard and decisions

Fill in the table from what you observed, not from reputation. Use 1 (hardest) to 5 (easiest) for the numeric column, and a short phrase elsewhere.

	Setup effort (1-5)	Filtering	Persistence	Hybrid effort	Would use for
FAISS					
Qdrant					
pgvector (optional)					

YOUR TURN Choose a system for each scenario and defend it in two sentences: (a) a weekend prototype in a notebook, (b) a company on PostgreSQL with 2 million support articles, (c) a public search service with strict per-customer filters and 200 million vectors.

Reflection questions

- Which operations were impossible or awkward in FAISS, and which of them would your application need?
- Why does Qdrant's COSINE collection accept our normalized vectors, and what would change with the dot-product distance?
- What would you do to avoid being locked into one database? (Lecture 5, abstraction layer.)
- Your results were measured on 30 documents. Which of your conclusions would you not trust at 30 million, and how would you test them?

Deliverable

Submit the executed notebook, the completed scorecard, the answers to each YOUR TURN box, and short answers to the reflection questions.

Troubleshooting

- **ImportError for qdrant_client:** re-run Cell 1 and restart the runtime if Colab asks.
- **Qdrant says the collection exists:** create a new client or call client.delete_collection('help') first.
- **pgvector build fails:** check the installed PostgreSQL major version and the release tag; the optional part may need adjusting.
- **Qdrant local mode is slow on large data:** expected. It is for learning the API, not for benchmarks.